讨论 PPT（1）

时钟之迹

从代码中追踪 STM32F407 的时钟：初始化、设置与获得

智装备 241　彭俊杰　2024306100324

例程参考：CH5EX01STD（标准外设库）/ CH5EX01HAL（硬件抽象库）

作业题目 • 追踪路线

四问对应四站，代码为准

① **SYSCLK 等时钟的初始化、设置和获得之迹**

STD 寄存器级 → 库函数 → HAL 三层代码逐级追踪（S4–S7）

② **SysTick 滴答时长设置，Delay / HAL_Delay 的调用方法**

轮询 COUNTFLAG 法 vs 中断 uwTick 法（S8–S10）

③ **画出流程图说明上述过程**

双库链路全景流程图（S11）

④ **总结 STM32 各时钟的原理以及应用的方法**

六源四钟 + 应用方法一览（S12）

依据 • 04 课件 P15–17　依据 • 03 课件 P26　例程 • CH5EX01STD / CH5EX01HAL

题目原文：①SYSCLK 等时钟的初始化、设置和获得之迹；②设置 SysTick 的滴答时长，调用 Delay 函数和 HAL_Delay 函数的方法；③画出流程图（伪代码也行）；④总结 STM32 微控制器各时钟的原理以及应用的方法

SYSCLK 从哪来：公式、时钟树与三选一

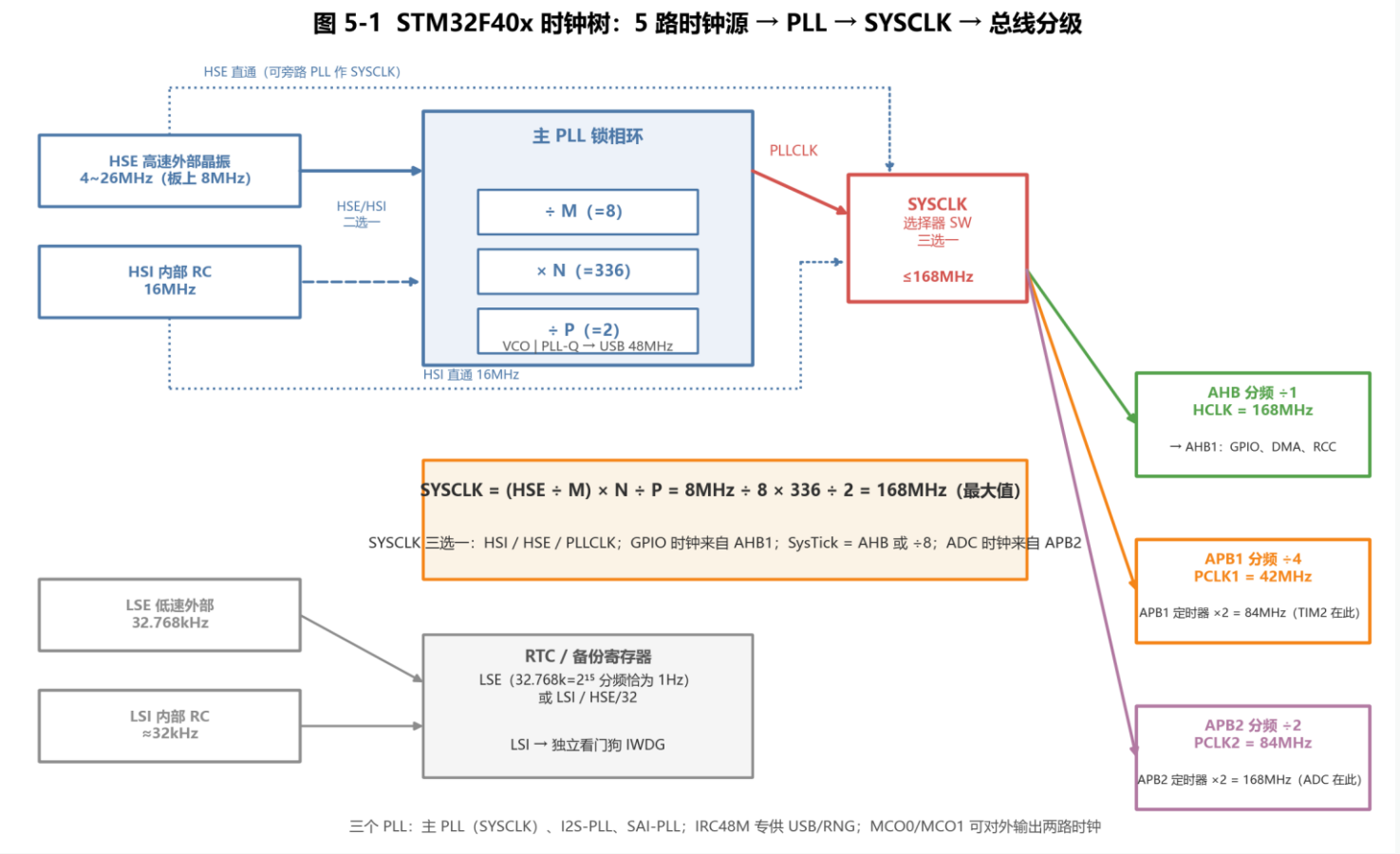
SYSCLK = (HSE / M) × N / P

= 8M / 8 × 336 / 2 = 168 MHz

HCLK 最大 168 MHz（AHB ÷1）
PCLK1 = APB1 最大 42 MHz，定时器 84 MHz
PCLK2 = APB2 最大 84 MHz，定时器 168 MHz

SYSCLK 三选一：HSI / HSE / PLLCLK

六个时钟源：HSE(4~26M) / LSE(32.768k) / HSI(16M) / LSI(32k) / IRC48M + 三个 PLL；
MCO0/1 两路输出
GPIO 时钟来源就是 AHB1 提供
SysTick 时钟是 AHB 分频 8 倍或直接得到
ADC 的时钟由 APB2 提供



图：STM32F40x 时钟树（5 路时钟源 → PLL → SYSCLK → 总线分级）

来源：《04--STM32芯片应用基础》教师版课件 slide 15–17（问题 4：时钟体系）

上电后谁配的时钟：Reset_Handler → SystemInit

```
// startup_stm32f40_41xxx.s —— 复位入口（向量表第 1 项）
Reset_Handler   PROC
                LDR     R0, =SystemInit
                BLX     R0              ; 先配时钟
                LDR     R0, =__main
                BX      R0              ; 再进 C 库→main

// system_stm32f4xx.c（CMSIS）—— SystemInit()
void SystemInit(void)
{
  RCC->CR |= (uint32_t)0x00000001;   // 开 HSI 兜底
  RCC->CFGR = 0x00000000;            // 复位时钟配置
  RCC->PLLCFGR = 0x24003010;         // 复位 PLL 参数
  SetSysClock();           // ★ 真正配置系统时钟
  SCB->VTOR = FLASH_BASE | VECT_TAB_OFFSET;
}

// SetSysClock() 内部（寄存器级五步）
RCC->CR |= RCC_CR_HSEON;             // ① 开 HSE
while(HSERDY==0);                    // ② 等就绪
RCC->CFGR |= HPRE_DIV1|PPRE2_DIV2|PPRE1_DIV4;
RCC->PLLCFGR = PLL_M|(PLL_N<<6)|(((PLL_P>>1)-1)<<16)
             |RCC_PLLCFGR_PLLSRC_HSE|(PLL_Q<<24);
RCC->CR |= RCC_CR_PLLON;  while(!PLLRDY);
RCC->CFGR |= RCC_CFGR_SW_PLL;        // ③ 切 PLL 作 SYSCLK
while(SWS != PLL);                   // ④ 等切换完成
```

1. **复位 RCC**
   回到已知状态（HSI 16M 兜底）
2. **开 HSE 并等待就绪**
   外部 8M 晶振起振
3. **设分频**
   AHB÷1 / APB1÷4 / APB2÷2
4. **配 PLL**
   M=8, N=336, P=2 → 168M
5. **切换 SYSCLK**
   SW=PLL，等 SWS 确认

代码来源：CH5EX01STD 工程 startup_stm32f40_41xxx.s + Libraries/CMSIS/system_stm32f4xx.c（SetSysClock 摘要）

RCC_HSE_Config：一行改 PLL，频率随参数走

```
// 5-STM32时钟系统实验 User/main.c（标准库函数法）
void RCC_HSE_Config(u32 pllm, u32 plln, u32 pllp, u32 pllq)
{
  RCC_DeInit();                        // 寄存器恢复默认
  RCC_HSEConfig(RCC_HSE_ON);           // 开 HSE
  if (RCC_WaitForHSEStartUp()==SUCCESS) {
    RCC_HCLKConfig(RCC_SYSCLK_Div1);   // HCLK  = SYSCLK/1
    RCC_PCLK2Config(RCC_HCLK_Div2);    // PCLK2 = HCLK/2
    RCC_PCLK1Config(RCC_HCLK_Div4);    // PCLK1 = HCLK/4
    RCC_PLLConfig(RCC_PLLSource_HSE,
                  pllm, plln, pllp, pllq);
    RCC_PLLCmd(ENABLE);
    while(RCC_GetFlagStatus(RCC_FLAG_PLLRDY)==RESET);
    RCC_SYSCLKConfig(RCC_SYSCLKSource_PLLCLK);
    while(RCC_GetSYSCLKSource()!=0x08); // 0x08=PLL 已当选
  }
}

int main(void)
{
  RCC_HSE_Config(8, 336, 2, 7);    // → 168 MHz
  ...                               // LED 闪烁验证
}
```

同一棵树的两种写法

寄存器级：RCC->CFGR / RCC->PLLCFGR
库函数级：RCC_xxxConfig() 一一封装
参数即公式：pllm/plln/pllp 对应 M·N·P

实验实证：改 P 变频

RCC_HSE_Config(8,336,4,7) → 84 MHz
LED 闪烁间隔随之加倍，肉眼可辨——
『时钟被重新设置』的最直接证据

代码来源：普中 T100 例程『5-STM32时钟系统』main.c（与 CH5EX01STD 同源的标准库工程结构）

main 的前两行：HAL_Init → SystemClock_Config

```
// CH5EX01HAL main.c（CubeMX 生成框架）
int main(void)
{
  HAL_Init();                  // ① 初始化中断/SysTick 时基
  SystemClock_Config();        // ② 配置系统时钟 168 MHz
  MX_GPIO_Init();
  while (1) { ... }
}

static void SystemClock_Config(void)
{
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 8;              // 8M ÷ 8
  RCC_OscInitStruct.PLL.PLLN = 336;            // ×336=VCO
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2; // ÷2=168M
  HAL_RCC_OscConfig(&RCC_OscInitStruct);      // 振荡器+PLL

  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider  = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
  HAL_RCC_ClockConfig(&RCC_ClkInitStruct,
                      FLASH_LATENCY_5);       // 同时配 Flash
}
```

结构体→寄存器

RCC_OscInitTypeDef / RCC_ClkInitTypeDef 两个字段组，对应 PLLCFGR 与 CFGR

两步走

HAL_RCC_OscConfig 管『源』，HAL_RCC_ClockConfig 管『分频+切换』

多了一件事

FLASH_LATENCY_5：168M 下 Flash 要 5 个等待周期，漏配会跑飞

同源殊途

与 STD 的 SetSysClock 完全同构，只是把寄存器操作藏进 HAL

代码来源：ST 官方 STM32CubeF4 / stm32f4xx-hal-driver（CH5EX01HAL 采用的 HAL 框架，STM324xG_EVAL 同型工程）

之迹① 获得 · 读回验证

时钟不是配完就算：RCC_GetClocksFreq 反算实频

```
// stm32f4xx_rcc.c —— 读寄存器反算当前各路频率
void RCC_GetClocksFreq(RCC_ClocksTypeDef* RCC_Clocks)
{
  tmp = RCC->CFGR & RCC_CFGR_SWS;   // 现在谁在当 SYSCLK?
  case 0x08:  /* PLL 作 SYSCLK */
    pllm = RCC->PLLCFGR & RCC_PLLCFGR_PLLM;
    pllvco = (HSE_VALUE / pllm)
           * ((RCC->PLLCFGR & RCC_PLLCFGR_PLLN) >> 6);
    pllp = (((RCC->PLLCFGR& RCC_PLLCFGR_PLLP)>>16)+1)*2;
    RCC_Clocks->SYSCLK_Frequency = pllvco / pllp;   // 168000000
}

// 使用（STD）                        // 使用（HAL）
RCC_GetClocksFreq(&clk);              f = HAL_RCC_GetSysClockFreq();
SystemCoreClockUpdate();              h = HAL_RCC_GetHCLKFreq();
```

本质：配置写寄存器，获得 = 读寄存器 + 按公式反算

读回

SWS 位看在用哪个源；PLLCFGR 反算频率（代码如左）

看灯

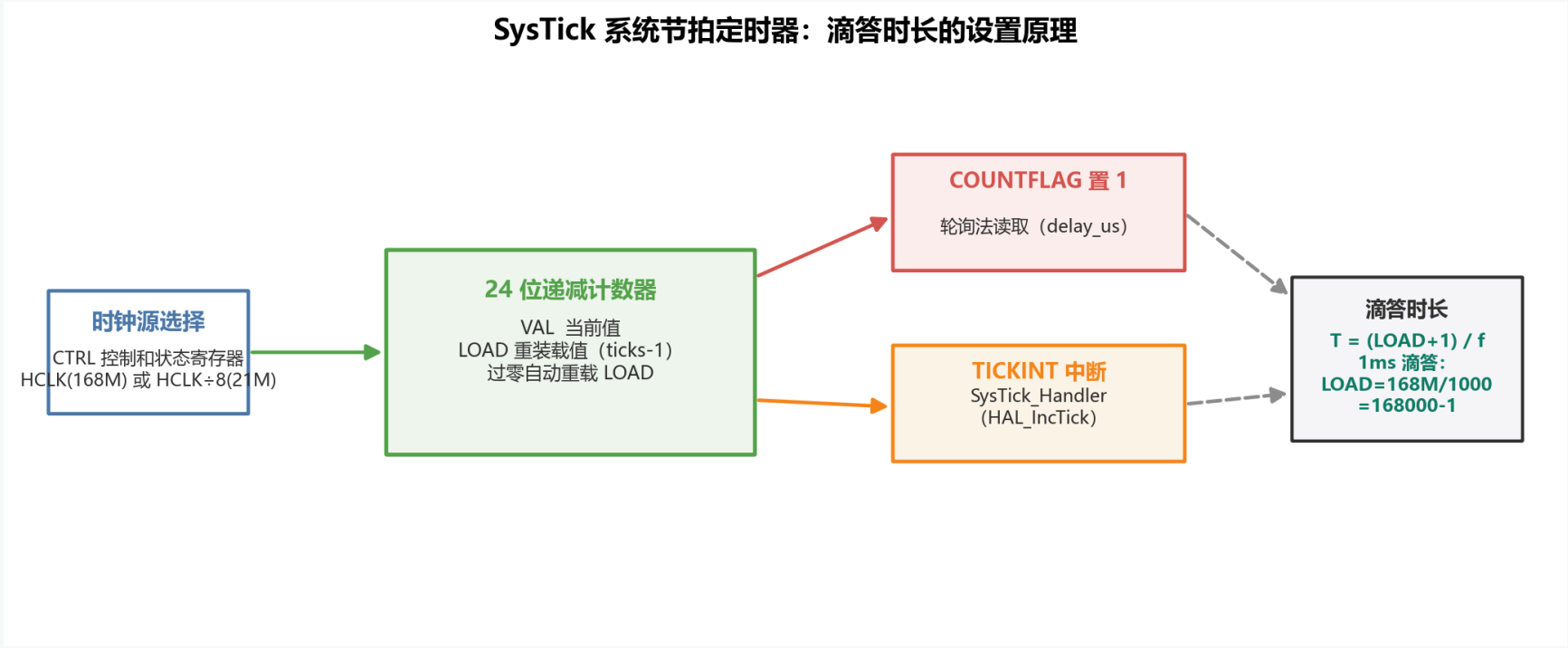
改 P=4 降频一半，LED 周期肉眼翻倍（5 页实验）

测量

MCO1(PA8) 输出 SYSCLK/2，示波器一量便知 84 MHz

代码来源：CH5EX01STD Libraries/STM32F4xx_StdPeriph_Driver/src/stm32f4xx_rcc.c（RCC_GetClocksFreq 摘要）

24 位倒计时定时器：三个寄存器定节拍




```
// core_cm4.h (CMSIS)
SysTick_Config(
  ticks) {
  LOAD = ticks-1;
  VAL  = 0;
  CTRL = CLKSOURCE
       | TICKINT
       | ENABLE;
}

// 1ms 滴答：
// ticks = 168M/1000
//       = 168000
```

滴答时长 T = (LOAD+1) / f　——f 取 HCLK(168M) 或 HCLK÷8(21M)，由 CTRL 的 CLKSOURCE 位选择

控制和状态 CTRL｜重载值 LOAD｜当前值 VAL：写过零自动重载、写入即清零（03 课件 P26）

依据：《03--Cortex-M4 内核体系结构》教师版课件 slide 26（系统定时器 SysTick 原文）；代码：CMSIS core_cm4.h

delay_us/delay_ms：轮询 COUNTFLAG 的阻塞延时



```c
// 7-SysTick系统定时器实验 Public/SysTick.c
void SysTick_Init(u8 SYSCLK)        // main 传 168
{
  SysTick_CLKSourceConfig(SysTick_CLKSource_HCLK_Div8);
  fac_us = SYSCLK/8;      // 21  → 1us 的计数次数
  fac_ms = fac_us*1000;   // 21000 → 1ms
}

void delay_us(u32 nus)
{
  SysTick->LOAD = nus * fac_us;     // 装载计数值
  SysTick->VAL  = 0x00;             // 写入即清零
  SysTick->CTRL |= 0x01;            // 开始倒数
  do {
    temp = SysTick->CTRL;
  } while((temp&0x01) && !(temp&(1<<16)));
  //          ↑使能位        ↑COUNTFLAG：数到 0 置 1
  SysTick->CTRL &= ~0x01;           // 关计数器
}

// main.c —— 调用方法
SysTick_Init(168);
while(1){ LED1=0; delay_ms(500); LED1=1; delay_ms(500); }
```

滴答时长的设置方法

时钟选 HCLK÷8 = 21 MHz

1μs → LOAD = 21

1ms → fac_ms = 21000

delay_us(n) → LOAD = n×21

24 位计数器上限 → 单次 ≤798 ms，delay_ms 用 540ms 分段凑

为什么叫轮询法

CPU 死等 CTRL 的第 16 位 COUNTFLAG 置 1——数到 0 硬件自动置位，不占用任何中断。

代码来源：普中 T100 例程『7-SysTick系统定时器』（与 CH5EX01STD 同源标准库工程）

每 1ms 一滴答：SysTick_Handler 累加 uwTick

```
// HAL_Init() 内部自动完成时基初始化
HAL_InitTick(TICK_INT_PRIORITY);
  HAL_SYSTICK_Config(SystemCoreClock/(1000U/uwTickFreq));
  //              ↑ 168M/1000=168000 → 恰好 1ms 滴答

// stm32f4xx_it.c —— 每 1ms 硬件自动进一次中断
void SysTick_Handler(void)
{
  HAL_IncTick();        // uwTick += uwTickFreq
}

// stm32f4xx_hal.c —— 延时 = 等滴答增量
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();  // 记起点
  uint32_t wait = Delay;
  wait += uwTickFreq;       // 补偿：保证最少延时
  while ((HAL_GetTick() - tickstart) < wait)
  { }                       // 空转，等中断把 uwTick 加够
}

// main.c —— 调用方法：HAL_Delay(500);   // 500ms
```

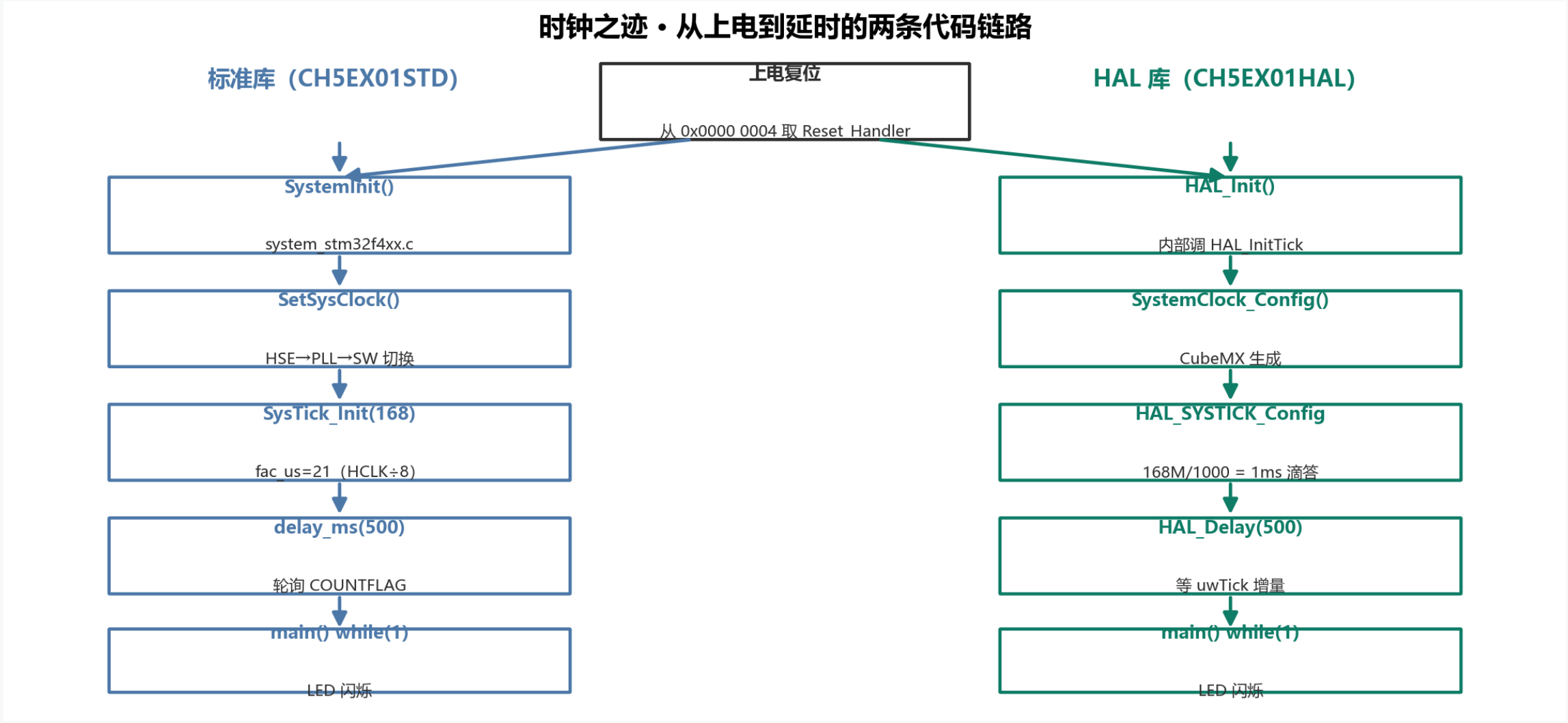
	Delay（STD 轮询）	HAL_Delay（HAL 中断）
时基来源	每次现装 LOAD	固定 1ms 滴答常开
到点判据	读 COUNTFLAG 位	uwTick 差值比较
中断占用	不占中断	每 1ms 进一次 ISR
附带能力	仅延时	全局时基：超时/RTOS 可改写 __weak

两法同源：都靠 SysTick 递减到 0；差别只在『谁去看它到 0』

代码来源：ST 官方 stm32f4xx-hal-driver（stm32f4xx_hal.c / stm32f4xx_it.c，CH5EX01HAL 同框架）

之迹③ 流程图

从上电到延时：两条链路一图看全



说明：左侧为 CH5EX01STD（标准外设库）链路，右侧为 CH5EX01HAL（硬件抽象库）链路；两者在『配置 168MHz → 配滴答 → 延时』三段同构

六源四钟：每条时钟的来路、去处与用法



时钟	来源 / 频率	去 处	应用方法（怎么用）
SYSCLK	三选一：HSI/HSE/PLL，最大 168 MHz	内核、总线基准	改 M/N/P 变频；SystemCoreClock 变量读频
HCLK	SYSCLK ÷1 = 168 MHz	AHB、DMA、GPIO(AHB1)	GPIO 翻转速度由它定；SysTick 可选源
PCLK1	HCLK ÷4 = 42 MHz	APB1 外设（TIM2…）	APB1 定时器自动×2=84M；外设库开时钟 RCC_APB1
PCLK2	HCLK ÷2 = 84 MHz	APB2 外设（USART1/ADC）	ADC 时钟由 APB2 提供；定时器×2=168M
HSE	外部晶振 4~26 MHz（板上 8M）	主 PLL 源	精度高作主时钟；失效可由 CSS 报警切 HSI
HSI	内部 RC 16 MHz	上电默认、PLL 备胎	不接晶振也能跑；省 BOM 但精度低
LSE / LSI	32.768k 晶振 / 内部 RC≈32k	RTC、IWDG	LSE 走 2^15 分频恰为 1Hz；LSI 喂独立看门狗
RTC/MCO	LSE/LSI/HSE÷32；MCO0/1 输出	日历 / 对外引脚 PA8	MCO 接示波器，验证真实时钟频率

依据：《04--STM32芯片应用基础》教师版课件 slide 15–17；实测口径与例程代码一致



三句话带走

时钟之迹 = 写寄存器 → 读寄存器

初始化/设置落在一组 RCC 寄存器上，获得就是读回反算（SWS + PLLCFGR）。

滴答只有一个公式

T = (LOAD+1)/f；Delay 轮询 COUNTFLAG，HAL_Delay 等 uwTick，同源不同看法。

例程即教材

CH5EX01STD 与 CH5EX01HAL 两条链路对照，公式、数字与老师课件 P15–17 完全一致。

谢谢聆听 · 智装备 241 彭俊杰 2024306100324